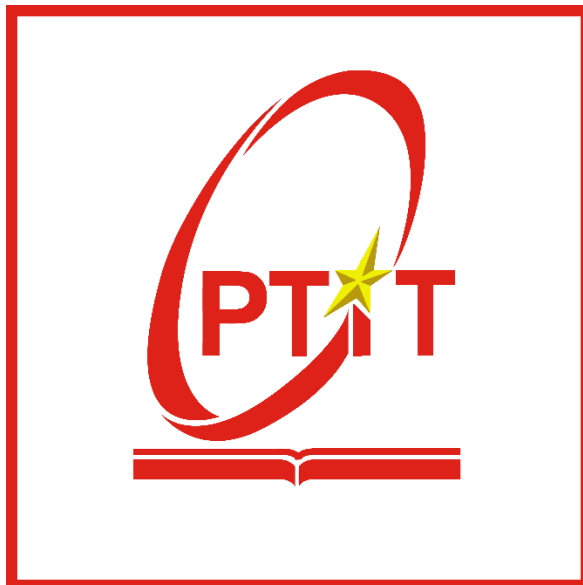


**HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN 1**



**Assignment 1
Môn học: Kiến trúc và Thiết kế phần mềm**

Giảng viên:	Trần Đình Quế
Học viên:	Nguyễn Đức Khải
Mã sinh viên:	B22DCCN441
Lớp:	E22CNPM01
Nhóm:	02

Hà Nội – 2025

Table of Contents

1. GIỚI THIỆU	2
1.1. Mô tả bài toán	2
1.2. Yêu cầu chức năng	2
1.3. Mục tiêu của đề tài	2
2. THIẾT KẾ CƠ SỞ DỮ LIỆU	2
3. TRIỂN KHAI PHIÊN BẢN 1: MONOLITHIC ARCHITECTURE.....	3
3.1. Khái niệm.....	3
3.2. Kiến trúc.....	3
3.3. Kết quả	4
4. TRIỂN KHAI PHIÊN BẢN 2: CLEAN ARCHITECTURE.....	4
4.1. Khái niệm.....	4
4.2. Cấu trúc Project.....	4
4.3. Điểm nổi bật.....	5
5. TRIỂN KHAI PHIÊN BẢN 3: MICROSERVICES ARCHITECTURE	5
5.1. Khái niệm.....	5
5.2. Chi tiết triển khai.....	6
5.3. Cơ chế giao tiếp	6
6. SO SÁNH VÀ ĐÁNH GIÁ	7
7. KẾT LUẬN.....	8
7.1. Kết quả đạt được	8
7.2. Bài học rút ra.....	8

1. GIỚI THIỆU

1.1. Mô tả bài toán

Trong bối cảnh thương mại điện tử phát triển mạnh mẽ, việc xây dựng một hệ thống website bán sách online là một bài toán phổ biến và thực tiễn. Hệ thống cho phép người dùng (khách hàng) có thể xem, lựa chọn và quản lý các sản phẩm sách trong giỏ hàng của mình một cách thuận tiện.

1.2. Yêu cầu chức năng

Bài tập lớn tập trung vào việc hiện thực hóa 4 chức năng cốt lõi trên cả 3 phiên bản kiến trúc:

1. Đăng ký và Đăng nhập: Cho phép khách hàng tạo tài khoản và xác thực để truy cập hệ thống.
2. Xem danh sách sách: Hiển thị danh mục các sản phẩm sách đang được bán.
3. Thêm sách vào giỏ hàng: Cho phép khách hàng thêm các cuốn sách mong muốn vào giỏ hàng cá nhân.
4. Xem chi tiết giỏ hàng: Hiển thị các sản phẩm, số lượng và thông tin liên quan trong giỏ hàng của khách hàng.

1.3. Mục tiêu của đề tài

Mục tiêu chính của bài tập lớn không chỉ là xây dựng một hệ thống hoàn chỉnh mà còn là:

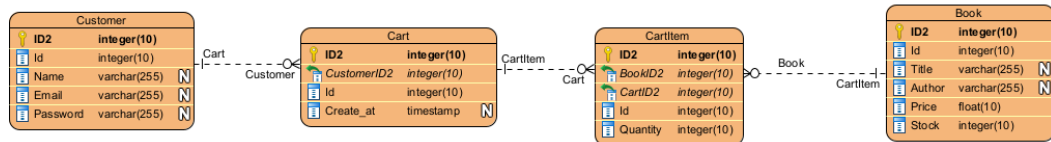
- Tìm hiểu sâu về ba mô hình kiến trúc phần mềm phổ biến: Monolithic, Clean Architecture và Microservices.
- Áp dụng lý thuyết vào thực tế bằng cách triển khai một hệ thống cụ thể trên cả ba kiến trúc này.
- Phân tích, so sánh và đánh giá ưu và nhược điểm của mỗi kiến trúc dựa trên các tiêu chí như độ phức tạp, khả năng bảo trì, mở rộng và hiệu năng. Qua đó, rút ra bài học kinh nghiệm về việc lựa chọn kiến trúc phù hợp cho từng loại dự án.

2. THIẾT KẾ CƠ SỞ DỮ LIỆU

Hệ thống sử dụng cơ sở dữ liệu quan hệ (MySQL) để lưu trữ dữ liệu. Các thực thể chính bao gồm:

- Customer: Lưu trữ thông tin về khách hàng, bao gồm thông tin đăng nhập và các thông tin cá nhân khác.
- Book: Lưu trữ thông tin chi tiết về từng cuốn sách như tên sách, tác giả, giá bán.

- Cart: Đại diện cho giỏ hàng của một khách hàng. Mỗi khách hàng có một giỏ hàng duy nhất.
- CartItem: Là một mục trong giỏ hàng, thể hiện một cuốn sách cụ thể được thêm vào giỏ với số lượng nhất định.



3. TRIỂN KHAI PHIÊN BẢN 1: MONOLITHIC ARCHITECTURE

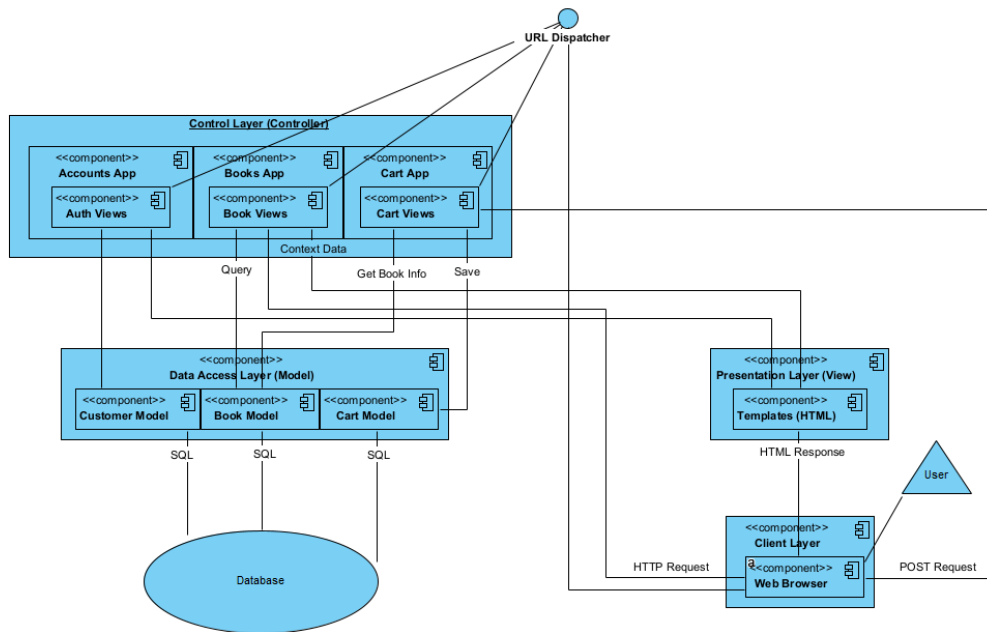
3.1. Khái niệm

Kiến trúc nguyên khối (Monolithic Architecture) là phương pháp phát triển phần mềm truyền thống, trong đó toàn bộ ứng dụng được xây dựng như một khối duy nhất. Tất cả các thành phần (giao diện người dùng, logic nghiệp vụ, xử lý dữ liệu) được đóng gói và triển khai chung trong một project, và thường sử dụng chung một cơ sở dữ liệu.

3.2. Kiến trúc

Phiên bản này được xây dựng dựa trên framework Django, tuân thủ theo mô hình Model-View-Template (MVT), một biến thể của mô hình Model-View-Controller (MVC).

- Model: Định nghĩa cấu trúc dữ liệu, tương tác trực tiếp với database.
- View: Chứa logic xử lý các yêu cầu HTTP và trả về các phản hồi.
- Template: Là lớp trình bày (Presentation Layer), chứa các file HTML để hiển thị dữ liệu cho người dùng.



3.3. Kết quả

Hệ thống phiên bản Monolithic đã được triển khai và chạy thành công trên Port 8000 của môi trường local.

4. TRIỂN KHAI PHIÊN BẢN 2: CLEAN ARCHITECTURE

4.1. Khái niệm

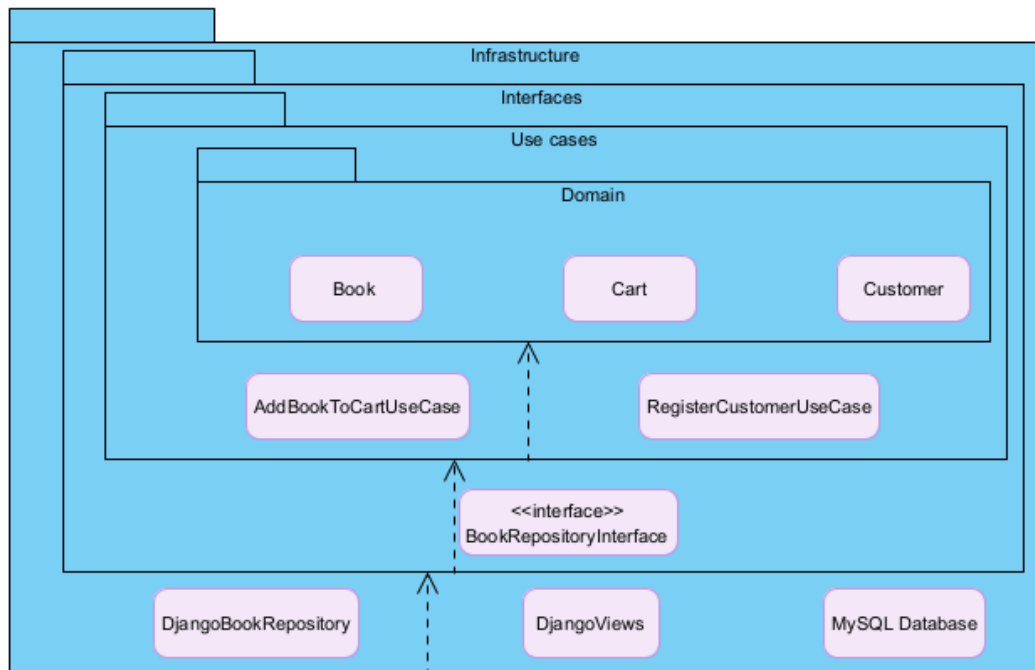
Clean Architecture là một triết lý thiết kế phần mềm tập trung vào việc phân tách các mối quan tâm (separation of concerns), giúp hệ thống trở nên độc lập với các chi tiết bên ngoài như framework, UI hay database. Nguyên tắc cốt lõi là Quy tắc Phụ thuộc (Dependency Rule): mã nguồn ở các tầng bên trong không được biết gì về các tầng bên ngoài.

4.2. Cấu trúc Project

Dự án được cấu trúc thành 4 tầng chính, đảm bảo sự độc lập và tuân thủ Dependency Rule:

- **Domain:** Tầng trong cùng, chứa các *Entities* (ví dụ: lớp Book, Cart) và các quy tắc nghiệp vụ cốt lõi nhất. Tầng này được viết bằng Python thuần, hoàn toàn không có sự phụ thuộc vào bất kỳ framework nào.
- **Use Cases (Application Business Rules):** Chứa logic nghiệp vụ của ứng dụng (ví dụ: AddBookToCartUseCase). Tầng này điều phối các entities để thực hiện các kịch bản sử dụng của người dùng.

- Interfaces (Adapter Layer): Định nghĩa các "công" (ports) hay các lớp trừu tượng (abstract classes) cho các thành phần bên ngoài, ví dụ như *Repository Interfaces*. Điều này giúp đảo ngược sự phụ thuộc.
- Infrastructure: Tầng ngoài cùng, là nơi triển khai các chi tiết cụ thể. Nó chứa các thành phần phụ thuộc vào framework như Django Models, Django Views, và các lớp triển khai cụ thể cho Repository Interfaces đã định nghĩa ở tầng Interfaces.



4.3. Điểm nổi bật

- Độc lập với Framework: Logic nghiệp vụ trong Use Cases và Domain không phụ thuộc vào Django, giúp dễ dàng thay đổi hoặc nâng cấp framework trong tương lai.
- Khả năng kiểm thử (Testability): Các quy tắc nghiệp vụ có thể được unit test một cách dễ dàng mà không cần khởi tạo toàn bộ ứng dụng Django, giúp tăng tốc độ và độ tin cậy của quá trình kiểm thử.

5. TRIỂN KHAI PHIÊN BẢN 3: MICROSERVICES ARCHITECTURE

5.1. Khái niệm

Kiến trúc Microservices chia nhỏ một ứng dụng lớn thành một tập hợp các dịch vụ nhỏ, độc lập. Mỗi dịch vụ chịu trách nhiệm cho một năng lực nghiệp vụ cụ thể, có mã nguồn,

quy trình triển khai và cơ sở dữ liệu riêng. Các dịch vụ giao tiếp với nhau qua các API được định nghĩa rõ ràng.

5.2. Chi tiết triển khai

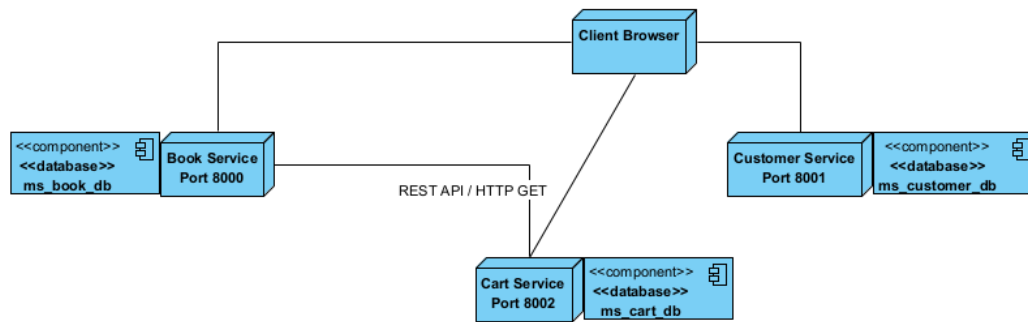
Hệ thống được tách thành 3 services độc lập:

- Service 1 - Book Service (Port 8000):
 - Chức năng: Quản lý thông tin sách (Thêm, xem danh sách, xem chi tiết).
 - Công nghệ: Django REST Framework.
 - Database: ms_book_db.
- Service 2 - Customer Service (Port 8001):
 - Chức năng: Quản lý người dùng (Đăng ký, đăng nhập, xác thực).
 - Công nghệ: Django REST Framework.
 - Database: ms_customer_db.
- Service 3 - Cart Service (Port 8002):
 - Chức năng: Quản lý giỏ hàng.
 - Công nghệ: Django REST Framework.
 - Database: ms_cart_db.

5.3. Cơ chế giao tiếp

Các service giao tiếp với nhau qua REST API. Một ví dụ điển hình là khi người dùng xem chi tiết giỏ hàng:

1. Client gửi yêu cầu đến Cart Service.
2. Cart Service lấy thông tin các CartItem từ database ms_cart_db.
Các CartItem này chỉ chứa book_id.
3. Để lấy thông tin chi tiết của sách (tên sách, giá), Cart Service thực hiện một cuộc gọi HTTP (GET request) đến API của Book Service (http://localhost:8000/api/books/<book_id>/).
4. Book Service trả về thông tin sách.
5. Cart Service tổng hợp thông tin và trả về kết quả cuối cùng cho client.



6. SO SÁNH VÀ ĐÁNH GIÁ

Tiêu chí	Monolithic Architecture	Clean Architecture	Microservices Architecture
Độ phức tạp khi code	Thấp: Dễ bắt đầu, code tập trung ở một nơi.	Cao: Đòi hỏi thiết kế kỹ lưỡng, nhiều lớp trừu tượng.	Rất cao: Phức tạp trong giao tiếp liên dịch vụ, quản lý data.
Khả năng bảo trì	Thấp: Khó thay đổi, các module liên kết chặt chẽ.	Cao: Các tầng độc lập, dễ dàng thay thế, sửa đổi.	Trung bình: Dễ bảo trì từng service nhưng khó debug tổng thể.
Khả năng mở rộng (Scalability)	Thấp: Phải scale toàn bộ ứng dụng, lãng phí tài nguyên.	Thấp: Tương tự Monolithic, scale toàn bộ hệ thống.	Cao: Có thể scale từng service riêng lẻ khi cần thiết.
Hiệu năng (Performance)	Cao: Giao tiếp nội bộ, không có overhead từ network.	Cao: Tương tự Monolithic.	Thấp hơn: Có độ trễ mạng (network latency) khi các service gọi nhau.
Triển khai (Deployment)	Đơn giản: Chỉ cần triển khai một project duy nhất.	Đơn giản: Tương tự Monolithic.	Phức tạp: Cần công cụ quản lý, giám sát nhiều dịch vụ.

Phân tích chi tiết:

- Monolithic Architecture rất phù hợp cho các dự án nhỏ, MVP (Minimum Viable Product) hoặc các đội ngũ nhỏ. Nó giúp phát triển nhanh ở giai đoạn đầu do sự đơn giản trong cấu trúc và triển khai. Tuy nhiên, khi hệ thống phình to, việc bảo trì, nâng cấp và mở rộng trở nên cực kỳ khó khăn. Một thay đổi nhỏ có thể ảnh hưởng đến toàn bộ hệ thống và việc áp dụng công nghệ mới là gần như không thể.
- Clean Architecture là một sự cải tiến của Monolithic, giải quyết bài toán về bảo trì và kiểm thử. Bằng việc phân tách rõ ràng các tầng và tuân thủ quy tắc phụ thuộc, nó giúp code trở nên linh hoạt, dễ đọc và dễ dàng thay đổi mà không làm ảnh hưởng đến logic nghiệp vụ cốt lõi. Dù vậy, nó vẫn là một khối thống nhất và gặp hạn chế về khả năng mở rộng độc lập như Monolithic.
- Microservices Architecture là lựa chọn tối ưu cho các hệ thống lớn, phức tạp và yêu cầu khả năng mở rộng cao. Việc chia nhỏ hệ thống giúp các đội nhóm có thể phát triển và triển khai các dịch vụ một cách độc lập, sử dụng các công nghệ khác nhau cho từng dịch vụ. Tuy nhiên, kiến trúc này đi kèm với sự phức tạp rất lớn trong việc thiết kế, triển khai, giám sát và đảm bảo tính nhất quán dữ liệu giữa các dịch vụ. Nó không phải là lựa chọn tốt cho các dự án nhỏ hoặc khi mới bắt đầu.

7. KẾT LUẬN

7.1. Kết quả đạt được

Qua quá trình thực hiện bài tập lớn, em đã hoàn thành các mục tiêu đề ra:

- Xây dựng thành công hệ thống bán sách online với đầy đủ 4 chức năng yêu cầu trên cả 3 phiên bản kiến trúc: Monolithic, Clean Architecture và Microservices.
- Toàn bộ mã nguồn của cả ba phiên bản đều hoạt động tốt, đáp ứng đúng các yêu cầu chức năng.
- Phân tích và so sánh được ưu, nhược điểm của từng kiến trúc dựa trên trải nghiệm lập trình thực tế.

7.2. Bài học rút ra

Sau khi hoàn thành đề tài, em đã rút ra được những bài học quý báu:

- Không có một kiến trúc nào là "tốt nhất" cho mọi bài toán. Việc lựa chọn kiến trúc phụ thuộc chặt chẽ vào bối cảnh dự án, quy mô hệ thống, yêu cầu về khả năng mở rộng, và nguồn lực của đội ngũ phát triển.

- Hiểu sâu về các nguyên tắc thiết kế như SOLID, Dependency Inversion (nền tảng của Clean Architecture) là cực kỳ quan trọng để xây dựng được phần mềm linh hoạt và dễ bảo trì.
- Kiến trúc Microservices mang lại nhiều lợi ích về khả năng mở rộng và sự độc lập, nhưng cũng đặt ra nhiều thách thức về "distributed computing" (tính toán phân tán) mà người lập trình phải giải quyết.
- Việc áp dụng lý thuyết vào thực hành giúp củng cố kiến thức một cách vững chắc và mang lại cái nhìn sâu sắc hơn về ngành khoa học máy tính và công nghệ phần mềm.